

Learning Objectives Quick Reference

Ghana AI Talent Accelerator - All 19 Notebooks

Speedykom Group GmbH

All 19 training notebooks now have clear learning objectives. Share these with trainers and students so everyone knows what to expect and what they'll learn.

Week 1: Welcome & Setup

Module 00: Why Python for AI & Data Science?

Students will learn: - Why Python dominates AI/Data Science (not JavaScript, Rust, or Go) - The “Triangle of Power” and when to use each language - Learning curves compared across languages - Why this career transition makes strategic sense

Module 01: Environment Setup (uv + VS Code)

Students will learn: - Install uv (Python package manager) - Understand virtual environments (prevent breaking system Python) - Create and manage Python projects - Install VS Code and configure for Python - Run Python code three ways: `uv run`, Jupyter Lab, manual activation - Install/manage dependencies safely - Understand `pyproject.toml` vs. `uv.lock`

Week 2: Version Control

Module 02: Professional Version Control (Git & GitHub)

Students will learn: - Difference between Git and GitHub - Initialize repositories and commit with meaningful messages - Create branches and merge code - Resolve merge conflicts - Push/pull from GitHub for collaboration - Version control for AI (experiment tracking) - Create professional portfolio on GitHub

Weeks 3-5: Python Basics (CRITICAL GATEWAY)

Module 03 Lesson 1: Variables & Data Structures

Students will learn: - Mindset shift from JavaScript to Python - The Big Four data structures (lists, tuples, dicts, sets) - Indexing and slicing (including negative indices) - Mutability and when to use each structure - F-strings for string formatting - Common mistakes when switching from JavaScript

Module 03 Lesson 2: Functions, Scope & Pythonic Patterns

Students will learn: - Write clean, reusable functions - Variable scope (LEGB rule) - Default arguments, *args, **kwargs - Type hints (optional but recommended) - Pythonic loops: `for...in`, `enumerate()`, `zip()` - List comprehensions (use them naturally!) - Python's truthiness model - Document code with docstrings

Module 03 Lesson 3: Comprehensions, Generators & Pythonic Patterns

Students will learn: - List, dict, set comprehensions - Generator expressions and lazy evaluation - Built-in functions: `map()`, `filter()`, `zip()`, `enumerate()`, `range()` - Recognize and write Pythonic idioms - Lambda functions (and when NOT to use them) - Exception handling (`try/except`) - Importing modules and organizing code

Weeks 6-8: Data Analysis with Pandas

Module 04 Lesson 1: Pandas Foundations

Students will learn: - DataFrames and Series (the fundamental structures) - Load data from CSV, JSON, Excel - Explore data (`.head()`, `.tail()`, `.shape`, `.dtypes`, `.info()`, `.describe()`) - Index with `.loc[]` (label-based) and `.iloc[]` (position-based) CONFUSING - Select columns and understand broadcasting - Sort, filter, unique values, value counts - Handle data types and convert - Identify data quality issues

Module 04 Lesson 2: Data Transformation & Quality

Students will learn: - Filter rows using boolean indexing and `.query()` - Create new columns through assignment - Handle missing data (`.fillna()`, `.dropna()`, `.interpolate()`) - Remove duplicates - Rename and reorder columns - Work with string columns (`.str` methods) - Use `.apply()` and `.map()` for custom transformations - Create derived features - Visualize data with Matplotlib/Seaborn - Tell data stories with clear visualizations

Module 04 Lesson 3: Groupby, Aggregation & Merging

Students will learn: - Use `.groupby()` to aggregate by categories - Aggregation functions (`.sum()`, `.mean()`, `.count()`, `.agg()`) - Create pivot tables and cross-tabs - Understand joins/merges (inner, outer, left, right) - Merge datasets on columns or indices - Reshape data (long wide with `.melt()` and `.pivot()`) - Set/reset index strategically

Weeks 9-10: Machine Learning (Compressed from 3 weeks)

Module 05 Lesson 1: ML Concepts & Linear Regression

Students will learn: - ML workflow: Problem → Data → Model → Evaluation → Deploy - Train/test split philosophy (WHY we don't train on test data) - Build Linear Regression models - Evaluate regression (MSE, RMSE, R^2 , MAE) - Identify overfitting - Preprocess features (scaling, encoding) - Supervised vs. unsupervised learning - Interpret model coefficients - Make predictions on new data

Module 05 Lesson 2: Classification & Decision Boundaries

Students will learn: - Classification vs. regression - Build classification models (Logistic Regression, Decision Trees, Random Forests) - Evaluate classification (Accuracy, Precision, Recall, F1, Confusion Matrix) - Understand class imbalance - Make predictions with probabilities - Tune decision thresholds - Visualize decision boundaries - Choose appropriate algorithms

Module 05 Lesson 3: Unsupervised Learning & Model Selection

Students will learn: - Unsupervised learning (find patterns without labels) - Clustering (K-Means, Hierarchical, DBSCAN) - Find optimal clusters (Elbow method, Silhouette score) - Dimensionality reduction (PCA) - Ensemble methods (why combining models is powerful) - Hyperparameter tuning (GridSearchCV, RandomizedSearchCV) - Cross-validation (K-fold) - Choose models strategically (interpretability vs. accuracy)

Weeks 11-12: Deep Learning & Advanced (Compressed from 3 weeks)

Module 06 Lesson 1: Neural Network Fundamentals

Students will learn: - Neural network architecture (layers, neurons, weights, biases) - Activation functions (ReLU, Sigmoid, Softmax) - Forward pass (how data flows through) - Backpropagation intuitively (not the math!) - Loss functions (cross-entropy, MSE) - Optimizers (SGD, Adam) - Identify overfitting/underfitting - Interpret learning curves - Why GPUs matter - Set up Google Colab

Module 06 Lesson 2: Building Neural Networks with PyTorch

Students will learn: - Build networks with `torch.nn.Module` - Define layers (Linear, Conv2D, RNN, Dropout, etc.) - Create training loops - Use GPU for training - Monitor training with loss plots - Save and load models - Convolutional layers (images) - Prevent overfitting (Dropout, L2 regularization) - Debug PyTorch errors - Transfer learning basics

Module 06 Lesson 3 & 07: Advanced Architectures & Capstone

Module 06 Lesson 3: Advanced Architectures & Transfer Learning

Students will learn: - Recurrent Neural Networks (RNNs) for sequences - Convolutional Neural Networks (CNNs) for images - Transfer learning: Use pre-trained models - Fine-tune on your own data - When to use transfer learning - Deploy models to production - Model compression (quantization, pruning) - Measure real-world performance - Address ethical concerns - Build end-to-end AI products

Module 07: Agentic AI & Orchestration Frameworks

Students will learn: - What Large Language Models (LLMs) are (pattern matching at scale) - What Retrieval Augmented Generation (RAG) is and how it overcomes hallucination - What orchestration frameworks do (coordinate multi-step workflows) - CodeTrain's three-stage Job pattern (receive_order → prepare_order → ship_order) - Job chaining with the » operator for sequential workflows - Conditional routing and branching (action » job syntax) - Self-looping for building interactive, stateful agents - The orchestration ecosystem: OpenCode, Claude Code, Anticravity, OpenClaw, CodeTrain, Langchain - When to use CodeTrain for learning vs. production frameworks for scale - How to build educational agents (chatbots, researchers, data analyzers) with CodeTrain - Python's role in AI orchestration (vs. TypeScript dominance in frameworks) - Why educational and production tools differ and when to transition

Quick Learning Gates (Where Students Get Stuck)

Gate 1: Week 5 (Python Mastery)

Success Criteria: - Write list comprehensions naturally - Understand function scope without confusion - Recognize Pythonic code patterns - **If <80% succeed, slow down and remediate**

Gate 2: Week 8 (Pandas Fluency)

Success Criteria: - Use .loc[] and .iloc[] automatically (not thinking about it) - Group by categories and aggregate - Answer data questions without looking up syntax - **If <80% succeed, extra practice before ML**

Gate 3: Week 12 (Capstone Completion)

Success Criteria: - Working AI project - Real data + real model + real deployment/demo
- Technical documentation - Can pitch to investors/stakeholders - **This is their portfolio piece**

For Trainers: How to Use These Objectives

1. **Start each notebook** by reviewing learning objectives with students
 2. **Assess at the end** using these objectives as a checklist
 3. **Identify struggling students** early (before they fall behind)
 4. **Flag knowledge gates** (Weeks 5, 8, 12) for remediation
 5. **Share with students** so they know what to expect
 6. **Iterate** based on what actually happens in class
-

Notes for Trainers

- **Module 03 is critical:** Everything after depends on Python fluency. If students aren't fluent here, they'll struggle for 8 more weeks.
 - **Pandas `.loc[]` vs `.iloc[]` confusion:** Universal problem. Plan extra time here.
 - **Deep Learning is abstract:** Students need hands-on repetition. Live demos are essential.
 - **Capstone is make-or-break:** This is what employers will see. Ensure it's real and portfolio-ready.
-

File Locations

All objectives have been added directly to the .qmd files. They appear at the top of each notebook (after the YAML frontmatter) as a “Learning Objectives” section.

Format used:

`## Learning Objectives`

`At the end of this notebook, you will be able to:`

- `- **Objective 1** - Clear, specific outcome`
- `- **Objective 2** - What students can actually do`
- `- ...`

Status: All 19 notebooks updated with learning objectives